



Numerical Optimization

Winter semester 2025/26

Exercise sheet 6 for Numerical Optimization

Manuel Caipo

Degree program: M.Sc. Computational Science Engineering

Enrollment number: 30783212

Instructors: Dr. Michael Lehn, Tobias Born

25. November 2025 - 89081 Ulm

Chapter 1

Solutions Exercise sheet 6

1.1 Exercise 1

1.1.1 Part a)

Showing that Algorithm 2 Reduces to Algorithm 1 for Quadratic Functions In this section we show that the nonlinear conjugate gradient method (Algorithm 2) becomes exactly the linear conjugate gradient method (Algorithm 1) when the objective function is quadratic of the form

$$f(x) = \frac{1}{2}x^T Ax - b^T x, \quad (1.1)$$

and when the step size strategy used in Algorithm 2 produces the optimal step size for quadratic functions.

The Gradient of the Quadratic Function

For the function

$$f(x) = \frac{1}{2}x^T Ax - b^T x, \quad (1.2)$$

the gradient is given by

$$\nabla f(x) = Ax - b. \quad (1.3)$$

Recall that Algorithm 1 defines the residual as

$$r_k = b - Ax_k. \quad (1.4)$$

Combining both relations gives

$$\nabla f(x_k) = Ax_k - b = -r_k. \quad (1.5)$$

Thus, the gradient and the residual differ only by a sign:

$$\boxed{\nabla f_k = -r_k}. \quad (1.6)$$

This identity will be used repeatedly in the following steps.

Correspondence of Initializations

Algorithm 2 initializes the direction as

$$d_0 = -\nabla f_0. \quad (1.7)$$

Using the identity $\nabla f_0 = Ax_0 - b = r_0$, we obtain

$$d_0 = -r_0, \quad (1.8)$$

which is exactly the initialization of Algorithm 1.

Thus, the initial search direction in both algorithms coincides.

Update of the Iterates

Both algorithms update the iterate using the same formula:

$$x_{k+1} = x_k + \alpha_k d_k. \quad (1.9)$$

Therefore, at this step there is no difference between the linear and nonlinear conjugate gradient methods.

Relationship Between Residuals and Gradients

In Algorithm 1, the residual is updated by

$$r_{k+1} = r_k + \alpha_k A d_k. \quad (1.10)$$

In Algorithm 2, the gradient at the next iterate is computed as

$$\nabla f_{k+1} = \nabla f(x_{k+1}) = A x_{k+1} - b. \quad (1.11)$$

Substituting $x_{k+1} = x_k + \alpha_k d_k$ yields

$$\nabla f_{k+1} = A(x_k + \alpha_k d_k) - b = (A x_k - b) + \alpha_k A d_k = \nabla f_k + \alpha_k A d_k. \quad (1.12)$$

Using $\nabla f_k = -r_k$ and $\nabla f_{k+1} = -r_{k+1}$, we rewrite the previous expression as

$$-r_{k+1} = -r_k + \alpha_k A d_k, \quad (1.13)$$

which is equivalent to the linear CG update

$$r_{k+1} = r_k + \alpha_k A d_k. \quad (1.14)$$

Thus, the gradient update in Algorithm 2 is exactly the same as the residual update in Algorithm 1.

Correspondence of the β -Formula (Fletcher–Reeves)

Algorithm 1 uses

$$\beta_{k+1} = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}. \quad (1.15)$$

Algorithm 2 (Fletcher–Reeves) uses

$$\beta_{k+1}^{FR} = \frac{\nabla f_{k+1}^T \nabla f_{k+1}}{\nabla f_k^T \nabla f_k}. \quad (1.16)$$

Using the identity $\nabla f_k = -r_k$, we obtain

$$\nabla f_k^T \nabla f_k = r_k^T r_k, \quad (1.17)$$

and similarly

$$\nabla f_{k+1}^T \nabla f_{k+1} = r_{k+1}^T r_{k+1}. \quad (1.18)$$

Thus,

$$\beta_{k+1}^{FR} = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k} = \beta_{k+1}. \quad (1.19)$$

Hence, for quadratic functions, the Fletcher–Reeves formula exactly reproduces the linear CG factor.

Update of the Search Direction

Algorithm 1 updates the direction by

$$d_{k+1} = -r_{k+1} + \beta_{k+1}d_k. \quad (1.20)$$

Algorithm 2 performs the direction update as

$$d_{k+1} = -\nabla f_{k+1} + \beta_{k+1}^{FR} d_k. \quad (1.21)$$

Using $\nabla f_{k+1} = -r_{k+1}$ and $\beta_{k+1}^{FR} = \beta_{k+1}$ gives

$$d_{k+1} = -r_{k+1} + \beta_{k+1}d_k, \quad (1.22)$$

which is exactly the direction update in Algorithm 1.

The Step Size α_k Coincides

Algorithm 1 uses the optimal step size

$$\alpha_k = \frac{r_k^T r_k}{d_k^T A d_k}. \quad (1.23)$$

Algorithm 2 requires a “suitable step size strategy.” For a quadratic function, the optimal α_k is the minimizer of

$$\phi(\alpha) = f(x_k + \alpha d_k). \quad (1.24)$$

We compute:

$$\phi(\alpha) = \frac{1}{2}(x_k + \alpha d_k)^T A(x_k + \alpha d_k) - b^T(x_k + \alpha d_k). \quad (1.25)$$

Differentiating with respect to α :

$$\phi'(\alpha) = d_k^T(Ax_k - b) + \alpha d_k^T A d_k. \quad (1.26)$$

Since $Ax_k - b = \nabla f_k = -r_k$, this becomes

$$\phi'(\alpha) = -d_k^T r_k + \alpha d_k^T A d_k. \quad (1.27)$$

Setting the derivative to zero gives the minimizer:

$$0 = -d_k^T r_k + \alpha_k d_k^T A d_k. \quad (1.28)$$

Solving for α_k yields

$$\alpha_k = \frac{d_k^T r_k}{d_k^T A d_k}. \quad (1.29)$$

Using the identity

$$d_k^T r_k = -r_k^T r_k, \quad (1.30)$$

we obtain the linear CG stepsize:

$$\alpha_k = \frac{r_k^T r_k}{d_k^T A d_k}. \quad (1.31)$$

Thus, the step size in Algorithm 2 matches that of Algorithm 1 under the assumption of optimal line search.

For the quadratic function $f(x) = \frac{1}{2}x^T A x - b^T x$, using exact line search, every component of Algorithm 2—the gradient, the step size α_k , the factor β_k , the search direction d_k , and the update of the iterate x_k —coincides exactly with the corresponding component of Algorithm 1.

Therefore, we conclude that Algorithm 2 reduces to Algorithm 1 in the quadratic case.

1.1.2 Part b)

```
1
2 function X = nonlin_cg(x0, grad_f, direction, step_size, max_it, tol)
3
4     x = x0(:);
5     n = length(x);
6
7
8     X = zeros(n, max_it + 1);
9     X(:, 1) = x;
10
11     grad_old = grad_f(x);    % grad f_0
12     d = -grad_old;          % d_0 = -grad f_0
13
14     % counter
15     k = 0;
16
17     while (norm(grad_old, 2) > tol) && (k < max_it)
18
19         alpha = step_size(x, d);
20
21         x_new = x + alpha * d;
22
23         grad_new = grad_f(x_new);
24
25         d = direction(grad_new, grad_old, d);
26
27         k = k + 1;
28
29         X(:, k + 1) = x_new;
30
31         x = x_new;
32         grad_old = grad_new;
33     end
34
35     X = X(:, 1:(k + 1));
36 end
```

nonlin_cg

1.1.3 Part c)

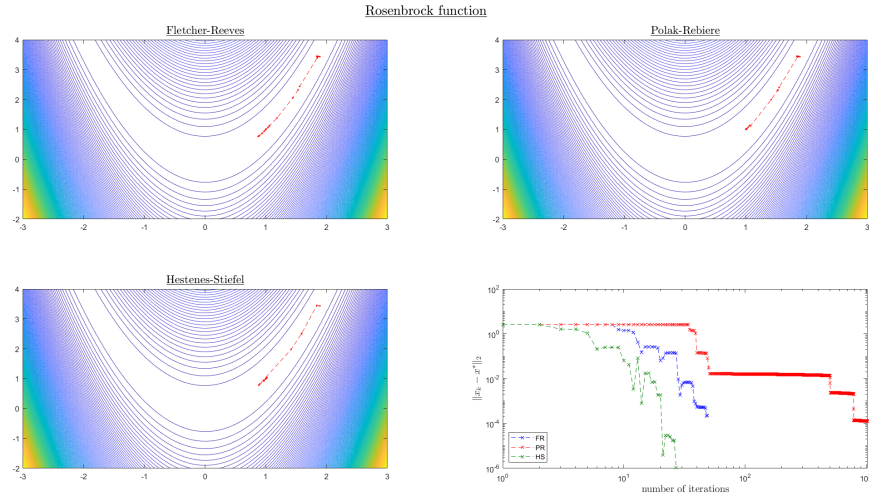


Figure 1.1: Convergence in rosenbrock

1.1.4 Part d)

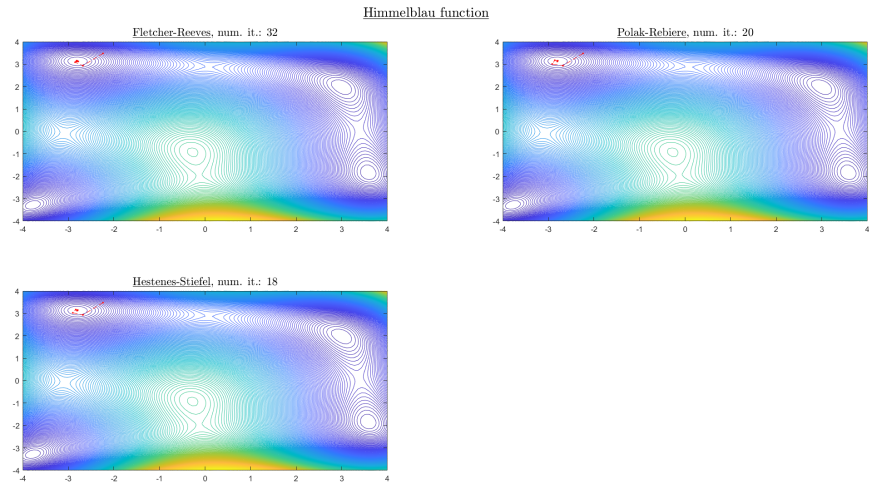


Figure 1.2: Convergence in himmelblau

In this project I implemented the nonlinear Conjugate Gradient (CG) method and compared the three classical update formulas—Fletcher–Reeves (FR), Polak–Rebriere (PR), and Hestenes–Stiefel (HS)—on the Rosenbrock and Himmelblau functions. All variants generate search directions of the form

$$p_k = -\nabla f(x_k) + \beta_k p_{k-1}. \quad (1.32)$$

$$\beta_k^{FR} = \frac{\|\nabla f(x_k)\|^2}{\|\nabla f(x_{k-1})\|^2}. \quad (1.33)$$

$$\beta_k^{PR} = \frac{\nabla f(x_k)^\top (\nabla f(x_k) - \nabla f(x_{k-1}))}{\|\nabla f(x_{k-1})\|^2}. \quad (1.34)$$

$$\beta_k^{HS} = \frac{\nabla f(x_k)^\top (\nabla f(x_k) - \nabla f(x_{k-1}))}{p_{k-1}^\top (\nabla f(x_k) - \nabla f(x_{k-1}))}. \quad (1.35)$$

The experiments show that all methods converge, but PR and especially HS require significantly fewer iterations than FR, demonstrating that the choice of β_k strongly affects convergence efficiency. The contour plots illustrate how CG efficiently navigates curved, narrow valleys (Rosenbrock) and finds local minima in nonconvex landscapes (Himmelblau). The main insights are that nonlinear CG critically depends on the update formula, a good line search is essential for stability, and PR/HS exploit curvature information more effectively, often achieving faster convergence than FR.

1.2 Exercise 2

1.2.1 Part a)

```

1
2 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
3 % f                function handle of function to be optimized
4 % x0              initial value
5 % T               parameter for random acceptance
6 % loc_opt         function handle for local optimizer
7 % h               maximum step size
8 % num_it          number of accepted steps
9 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
10
11 function[X] = basin_hopping(f, x0, T, loc_opt, h, num_it)
12
13 % initial local optimization
14 x_loc = loc_opt(x0);          % x_loc <- loc_opt(x0)
15 f_loc = f(x_loc);
16
17 X = [x0,x_loc];
18
19 % counter for accepted steps
20 i = 0;
21
22 % iterate until number of accepted steps reached
23 while i < num_it
24
25     % take random step from x_loc to x_trial
26     n = numel(x_loc);
27
28     % local optimization starting from x_trial
29     r = (2*rand(n,1) - 1) * h;
30     x_trial = x_loc + r;
31
32     x_trial_loc = loc_opt(x_trial); % x_trial_loc <- loc_opt(x_trial)
33     f_trial     = f(x_trial_loc);   % f_trial <- f(x_trial_loc)
34
35     X = [X,x_trial,x_trial_loc];
36
37     delta = f_trial - f_loc;
38
39     % accept step if improvement in function value or wrt a probability
40     if delta <= 0 || rand() < exp(-delta / T)
41
42         % x_trial_loc is new x_loc
43         x_loc = x_trial_loc;
44         f_loc = f_trial;
45
46         % increase counter of accepted steps
47         i = i + 1;
48
49     end
50
51 end
52
53 end

```

Basin hopping

1.2.2 Part b)


```

1
2 function xk = hooke_jeeves(f, x0, h0, eps)
3 % HOOKE_JEEVES HookeJeeves method (local optimization)
4 %
5 %   xk = hooke_jeeves(f, x0, h0, eps, max_it)
6 %
7 %   f       : handle to objective function f:R^n -> R
8 %   x0       : initial column vector
9 %   h0       : initial step size(s); can be scalar or vector
10 %   eps      : tolerance for ||h||_1
11 %   max_it   : maximum number of iterations (pattern moves)
12 %
13 %   xk       : approximation of local minimizer
14   max_it = 1000;
15
16   % ensure column vectors
17   xk = x0(:);
18   n = numel(xk);
19
20   % step vector h (if scalar is given, copy it to all coordinates)
21   if numel(h0) == 1
22       h = h0 * ones(n,1);
23   else
24       h = h0(:);
25   end
26
27   k = 0;
28
29   % main loop: Algorithm 2
30   while norm(h,1) > eps && k < max_it
31
32       % y^(k) <- EXPLORATION(f, x^(k), h)
33       yk = exploration(f, xk, h);
34
35       % if y^(k) == x^(k) (no improvement in exploration)
36       if all(yk == xk)
37           % h <- 1/2 h
38           h = 0.5 * h;
39       else
40           % w^(k) <- 2 y^(k) - x^(k)
41           wk = 2*yk - xk;
42
43           % z^(k) <- EXPLORATION(f, w^(k), h)
44           zk = exploration(f, wk, h);
45
46           % if f(z^(k)) < f(y^(k)) then x^(k+1) <- z^(k)
47           if f(zk) < f(yk)
48               xk = zk;
49           else
50               % else x^(k+1) <- y^(k)
51               xk = yk;
52           end
53       end
54
55       k = k + 1;
56   end
57 end
58
59
60 %-----
61 % Subfunction: Algorithm 3 (Exploration procedure)
62 %-----
63 function x = exploration(f, x, h)

```

```

64 % f : function handle
65 % x : current point (column)
66 % h : vector of step sizes
67
68 n = numel(x);
69
70 for j = 1:n
71     % try x + h_j e_j
72     x_trial = x;
73     x_trial(j) = x(j) + h(j);
74
75     if f(x_trial) < f(x)
76         x = x_trial;
77     else
78         % if no improvement, try x - h_j e_j
79         x_trial(j) = x(j) - h(j);
80         if f(x_trial) < f(x)
81             x = x_trial;
82         end
83     end
84 end
85 end

```

Hooke Jeeves

1.2.3 Part c)

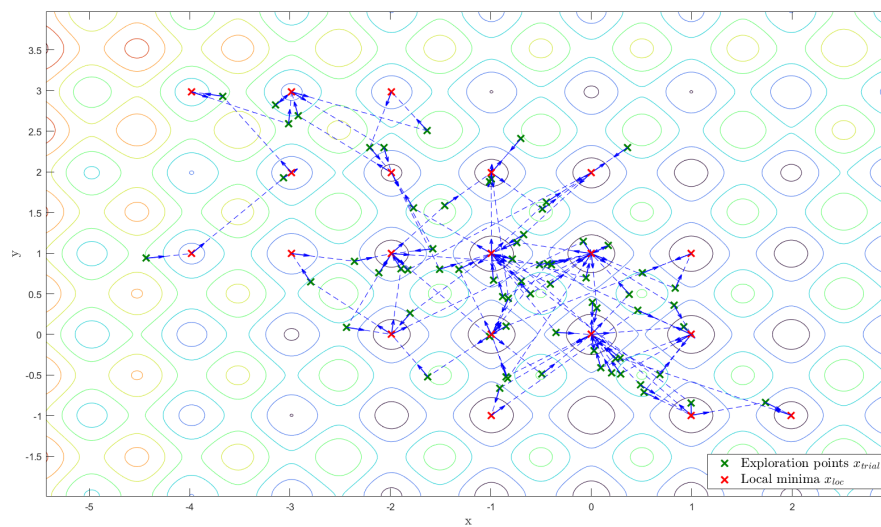
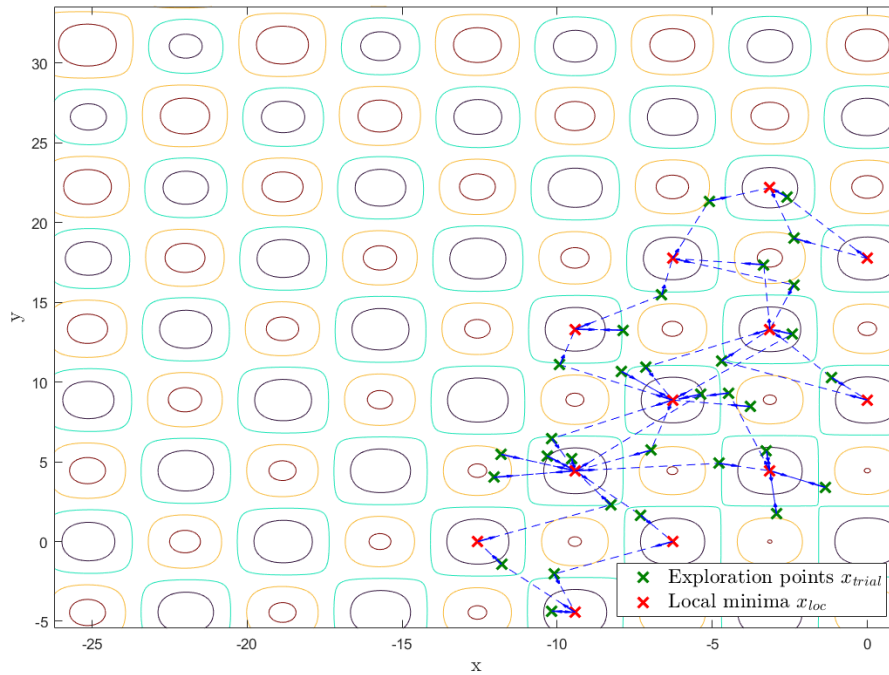


Figure 1.3: Basin hopping f1

1.2.4 Part d)

Figure 1.4: Basin hopping f_2

Basin hopping combines stochastic exploration with deterministic local minimization, turning any local optimizer into a global one. In the multimodal test functions f_1 and f_2 , the algorithm successfully identifies multiple distinct basins of attraction: each random perturbation generates an exploration point x_{trial} , which is then mapped to a nearby local minimum x_{loc} via a local optimizer. Unlike swarm-based metaheuristics, basin hopping does not rely on a population but on successive jumprelax cycles governed by a Metropolis-type acceptance rule. The experiments highlight that basin hopping efficiently escapes local minima and systematically explores the landscape, while its effectiveness depends strongly on the quality of the chosen local optimizer, making it particularly suitable for smooth, multimodal functions where gradient-free swarms are less efficient.

Exercise sheet 6 for Numerical Optimization

Dr. Michael Lehn

Tobias Born

Deadline: November 26, 2025

Exercise 1 (Non-linear cg-method, 7+13+4+4 points)

In the lecture *Numerical Linear Algebra*, we learned about the cg-method for solving linear equation systems

$$Ax = b \quad \text{with} \quad A \in \mathbb{R}^{n \times n}, b \in \mathbb{R}^n.$$

The aim of the cg-method was to minimize the quadratic function

$$f(x) = \frac{1}{2}x^T Ax - b^T x.$$

In the case of a positive definite matrix A , this optimization problem admits a unique solution that also is the solution to the system of linear equations.

The cg-method can be understood as a descent method. In each iteration step, the current (negative) residual is orthogonalized to the previous residuals and used as the search direction. As the function is quadratic, the line search for the step size can be solved exactly and the optimal step size can be used. If calculated exactly, the cg-method solves the quadratic optimization problem and thus the system of linear equations in n steps. Here the algorithm for the *linear cg-method*:

Algorithm 1: Linear cg-method

```
1 Input:  $A \in \mathbb{R}^{n \times n}$ ,  $b \in \mathbb{R}^n$ , initial value  $x_0 \in \mathbb{R}^n$ 
2 Output: approximation  $x_k$  of the solution to the system of linear equations  $x^* = A^{-1}b$ 
3  $r_0 \leftarrow Ax_0 - b$ 
4  $d_0 \leftarrow -r_0$ 
5  $k \leftarrow 0$ 
6 while  $\|r_k\|_2 \neq 0$  do
7    $\alpha_k \leftarrow \frac{r_k^T r_k}{d_k^T A d_k}$ 
8    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
9    $r_{k+1} \leftarrow Ax_{k+1} - b = r_k + \alpha_k A d_k$ 
10   $\beta_{k+1} \leftarrow \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$ 
11   $d_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} d_k$ 
12   $k \leftarrow k + 1$ 
```

Line 11 ensures that d_{k+1} is orthogonal to all previous search directions $d_1, \dots, d_k$ with respect to the inner product $\langle A \cdot, \cdot \rangle$. Thereby, line 11 corresponds to the Gram-Schmidt procedure, where the terms for all search directions $d_1, \dots, d_{k-1}$ turn out to be zero.

Now we want to generalize the approach from quadratic to general smooth functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$. This raises several problems. First, in the quadratic case the residual can be specified and corresponds to the gradient of f , since $f'(x) = Ax - b$. In the general case, the residual is unknown. We replace it with the gradient of f , but this no longer corresponds to the residual. This also means that the search directions are no longer orthogonal to each other. Second, the optimal

step size cannot be computed and a step size strategy (Armijo condition, Wolfe conditions, ...) has to be employed. The algorithm of the *non-linear cg-method* is:

Algorithm 2: Non-linear cg-method

```

1 Input: gradient of the function  $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ , initial value  $x_0 \in \mathbb{R}^n$ 
2 Output: approximation  $x_k$  of a solution to the optimization problem  $\min_x f(x)$ 
3 compute  $\nabla f_0 \leftarrow \nabla f(x_0)$ 
4  $d_0 \leftarrow -\nabla f_0$ 
5  $k \leftarrow 0$ 
6 while  $\|\nabla f_k\|_2 \neq 0$  do
7   compute step size  $\alpha_k$  with suitable step size strategy
8    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
9   compute  $\nabla f_{k+1} \leftarrow \nabla f(x_{k+1})$ 
10   $\beta_{k+1}^{\text{FR}} \leftarrow \frac{\nabla f_{k+1}^T \nabla f_{k+1}}{\nabla f_k^T \nabla f_k}$ 
11   $d_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{\text{FR}} d_k$ 
12   $k \leftarrow k + 1$ 

```

There is no reason to assume that this approach generally works well. The hope is that the function behaves like a quadratic function around a local minimum and that algorithm 2 behaves similarly to algorithm 1 there.

a) Show that algorithm 2 corresponds to algorithm 1 for quadratic functions

$$f(x) = \frac{1}{2}x^T A x - b^T x ,$$

assuming the step size strategy would yield the optimal step size for quadratic functions (algorithm 1, line 7).

Actually, algorithm 2 is called *non-linear cg-method according to Fletcher-Reeves*. The cg-method is generalized in various ways. The difference lies in how the factor β_{k+1} in line 11 is selected. For example, in the *non-linear cg-method according to Polak-Rebière*, it is chosen as

$$\beta_{k+1}^{\text{PR}} = \frac{\nabla f_{k+1}^T (\nabla f_{k+1} - \nabla f_k)}{\nabla f_k^T \nabla f_k}$$

and in the *non-linear cg-method according to Hestenes-Stiefel* it is

$$\beta_{k+1}^{\text{HS}} = \frac{\nabla f_{k+1}^T (\nabla f_{k+1} - \nabla f_k)}{d_k^T (\nabla f_{k+1} - \nabla f_k)} .$$

These two methods no longer correspond to the linear cg-method in the case of a quadratic function, but they perform better in other situations.

b) Create a MATLAB program `nonlin_cg.m` that implements algorithm 2. It should be handed

- an initial value `x0`,
- a function handle `grad_f` for the function's gradient,
- a function handle `direction(grad_f_curr, grad_f_old, d)` to determine the search direction d_{k+1} , where `grad_f_curr` is the evaluation of the gradient at x_{k+1} , `grad_f_old` the evaluation of the gradient at x_k and `d` the previous search direction d_k ,

- a function handle `step_size(x,d)` for the step size selection,
- a maximum number of iteration steps `max_it`
- as well as a tolerance `tol` for the termination criterion.

It is supposed to either terminate if $\|\nabla f(x_k)\|_2 < \text{tol}$ or if the maximum number of iteration steps is reached. Make it return a matrix of dimension $(n \times (\text{number iteration steps} + 1))$ containing all iterates.

- c) The MATLAB program `test_cg_rosenbrock.m` tests your non-linear cg-method on Rosenbrock's function using the Fletcher-Reeves variant, the Polak-Rebière variant and the Hestenes-Stiefel variant.

First, fill the gaps for all three function handles for the determination of the search direction. Then execute the program and reflect on the results.

- d) Do the same with `test_cg_himmelblau.m`.

Exercise 2 (Global optimization: basin hopping method, 13+3+3+3 points)

We have time for a second heuristic, global optimization approach. *Basin hopping* does exactly what the name suggests. This method attempts to jump from valley to valley on the topography of the function. In each valley, it then determines the smallest value using a local optimization approach. The goal is to find many local minima in order to obtain a value that globally is as small as possible.

The process is as follows: First, a local optimization is performed starting from the initial value. Then, starting from the local minimum, a step is taken in a random direction with the step size being random but bounded by a maximum step size. From this exploration point, again a local optimization is performed. The hope is that the exploration step reaches a new basin and therefore the local optimization can find a new local minimum. If this is successful, the iteration continues from the new local minimum. If this is not successful, a new exploration step is taken starting from the old local minimum. In order to create a certain degree of dynamics and randomness, an unsuccessful step is also accepted with a certain probability.

Algorithm 3: Basin hopping method

```
1 Input:  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ , initial value  $x_0 \in \mathbb{R}^n$ ,  $T > 0$ , a local optimization method  
    $\text{loc\_opt}(\mathbf{x})$ , step size  $h > 0$ , number iteration steps  $\text{num\_it}$   
2 Output: sequence of points  $x_{\text{loc}}$  trying to find as many local minima as possible  
3  $x_{\text{loc}} \leftarrow \text{loc\_opt}(x_0)$   
4  $f_{\text{loc}} \leftarrow f(x_{\text{loc}})$   
5  $i \leftarrow 0$   
6 while  $i < \text{num\_it}$  do  
7    $r \leftarrow (2 \cdot \text{rand}(n, 1) - 1) \cdot h$   
8    $x_{\text{trial}} \leftarrow x_{\text{loc}} + r$   
9    $x_{\text{trial\_loc}} \leftarrow \text{loc\_opt}(x_{\text{trial}})$   
10   $f_{\text{trial}} \leftarrow f(x_{\text{trial\_loc}})$   
11  if  $(f_{\text{trial}} - f_{\text{loc}} \leq 0) \vee$  with probability  $\exp\left(-\frac{f_{\text{trial}} - f_{\text{loc}}}{T}\right)$  then  
12     $x_{\text{loc}} \leftarrow x_{\text{trial\_loc}}$   
13     $f_{\text{loc}} \leftarrow f_{\text{trial}}$   
14     $i \leftarrow i + 1$ 
```

Here, $\text{rand}(n, 1)$ is an n -dimensional vector containing uniformly distributed random numbers between zero and one. Thus, r is a step in a random direction with a random step size, which is however bounded by h . The counter i does not count the number of iteration steps, but the number of accepted steps.

a) Implement the basin hopping method. Therefore, fill the gaps in `basin_hopping.m`.

For the local optimization method we will use our implementation of the Hooke-Jeeves method. We could use any of our local optimizers, but for the sake of simplicity, we use a direct search method so that we do not have to deal with convergence issues of the local optimizer.

b) Adapt your implementation of the Hooke-Jeeves method such that it can be used for basin hopping.

Now we are ready to test our basin hopping. We will use the same two test cases as for the

particle swarm optimization, namely

$$f_1(x, y) = x^2 + y^2 + 10 (2 - \cos(2\pi x) - \cos(2\pi y)) \quad \text{and}$$
$$f_2(x, y) = 1 + \frac{x^2 + y^2}{4000} - \cos(x) \cos\left(\frac{y}{\sqrt{2}}\right) .$$

- c) Execute `test_basin_hopping_f1.m` and reflect on the results.
- d) Execute `test_basin_hopping_f2.m` and reflect on the results.